

universität wien

Reactly

A ReAct-inspired task-splitting tool-using agent build from scratch for local open-weight LLMs

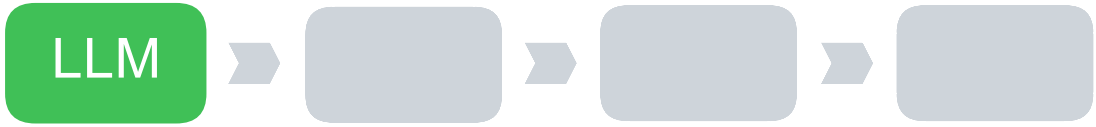
Bachelor Thesis by Donovan Käfer

Supervisor: Dipl.-Ing. Dr.techn Marian Lux

Backstory LLM

- Based on Transformer-architecture introduced in 2017 [1]
- Hugely successful
- Limitations:
  - Struggle with logic- & math-heavy tasks
  - Internal knowledge outdated and non-customized
  - usually performance-heavy
  - limited context window size

Fix #1: Reasoning



- Explicit intermediate reasoning
- Improves logic + math accuracy

Chain-of-Thoughts [2] (CoT)

- Wei et al. (2022)
- Uses in-context learning

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had 23 - 20 = 3. They bought 6 more apples, so they have 3 + 6 = 9. The answer is 9. ✓

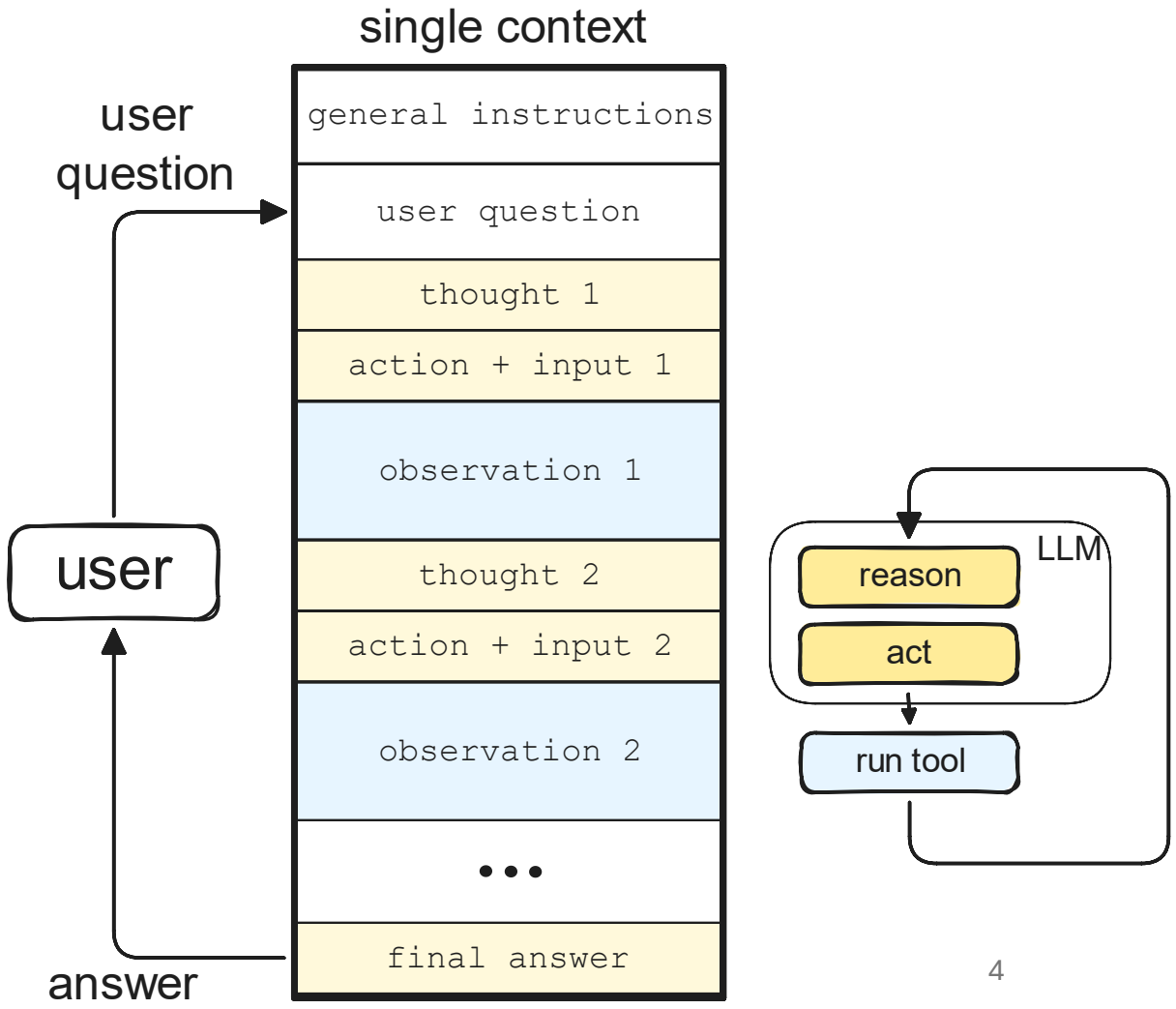
Figure 1 of CoT paper from Wei et al. (2022)

Fix #2: Agency

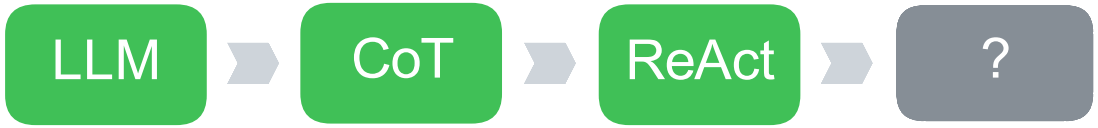


- Allow LLM to interact with environment
- Tools:
  - Provide external up-to-date information (e.g. web-search)
  - Act on user's behalf (e.g. for smart home)
  - Can offload work from the LLM (e.g. calculator)
- Combine reasoning + acting
  → ReAct [7]

ReAct loop



The Remaining Problem



- Struggle with logic-heavy tasks
- Knowledge outdated & non-specific
- Typically very performance heavy

- Limited context window size

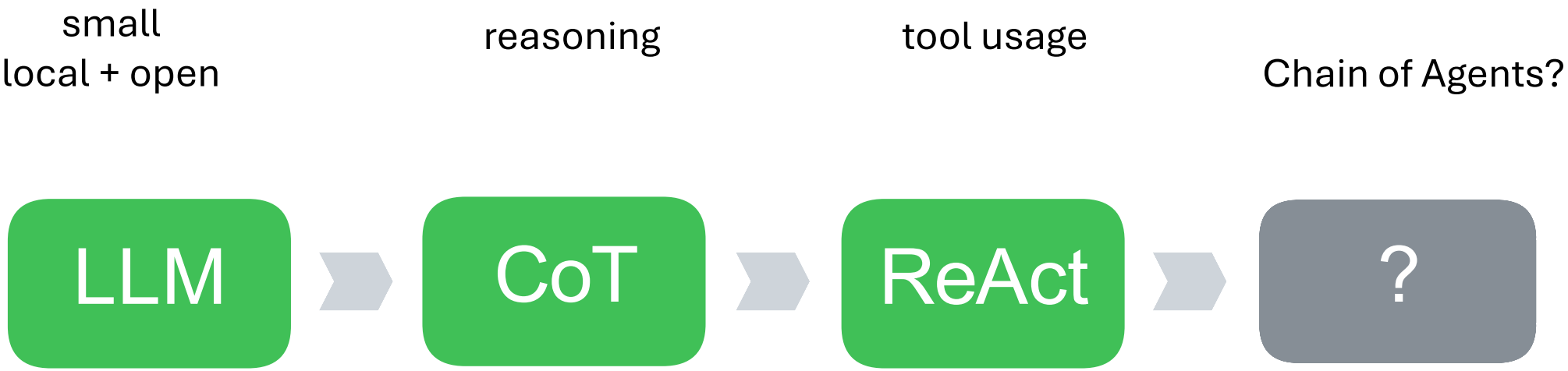
✅ CoT reasoning

✅ Acting (tools+APIs)

💡 reducing the input length
  - e.g. RAG

💡 expanding context window
  - loss-in-the-middle!

Solution?


small
local + open
reasoning
tool usage
Chain of Agents?
LLM
CoT
ReAct
?

Solution? Chain of Agents (Yusen Zhang et.al. 2024) [7]

- Splitting up long context into chunks
- Assigning each worker agent a chunk to process
- Lastly, a Manager Agent answers the question
- Only solves context size limitation

Solution? Chain of Agents (OPPO AI Agent Team)

- Single LLM is trained to role-play different sub-agents
- Eliminates inter-agent communication
- Requires to train a model → out of scope for this bachelor thesis

Solution? Plan-then-Solve

- Agent formulates a plan and then executes it
- Prompting technique: “Plan-and-Solve Prompting” by
- Workflow / Architecture: “Plan-then-Execute” [4]
  - **Planner**: generates plan (as sequence, DAG, …) from query
  - **Executor**: executes plan step by step
  - Optionally: **Verifier**, **Refiner**

Reactly

- ReAct-inspired
- task-splitting
- tool-using agent
- build from scratch
- for local open-weight LLMs

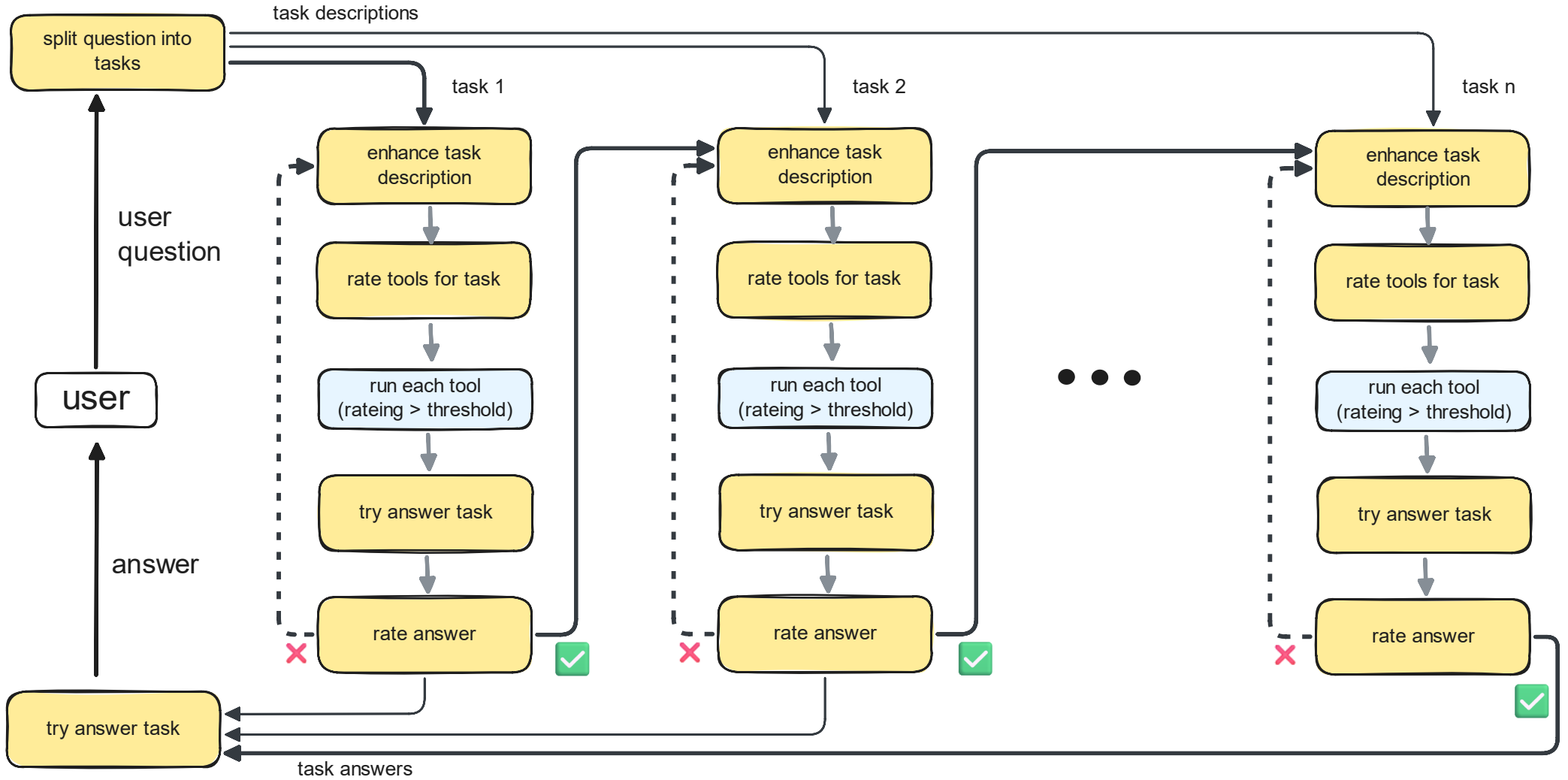
→ see architecture on next slide

→ Similar API to LLamaIndex

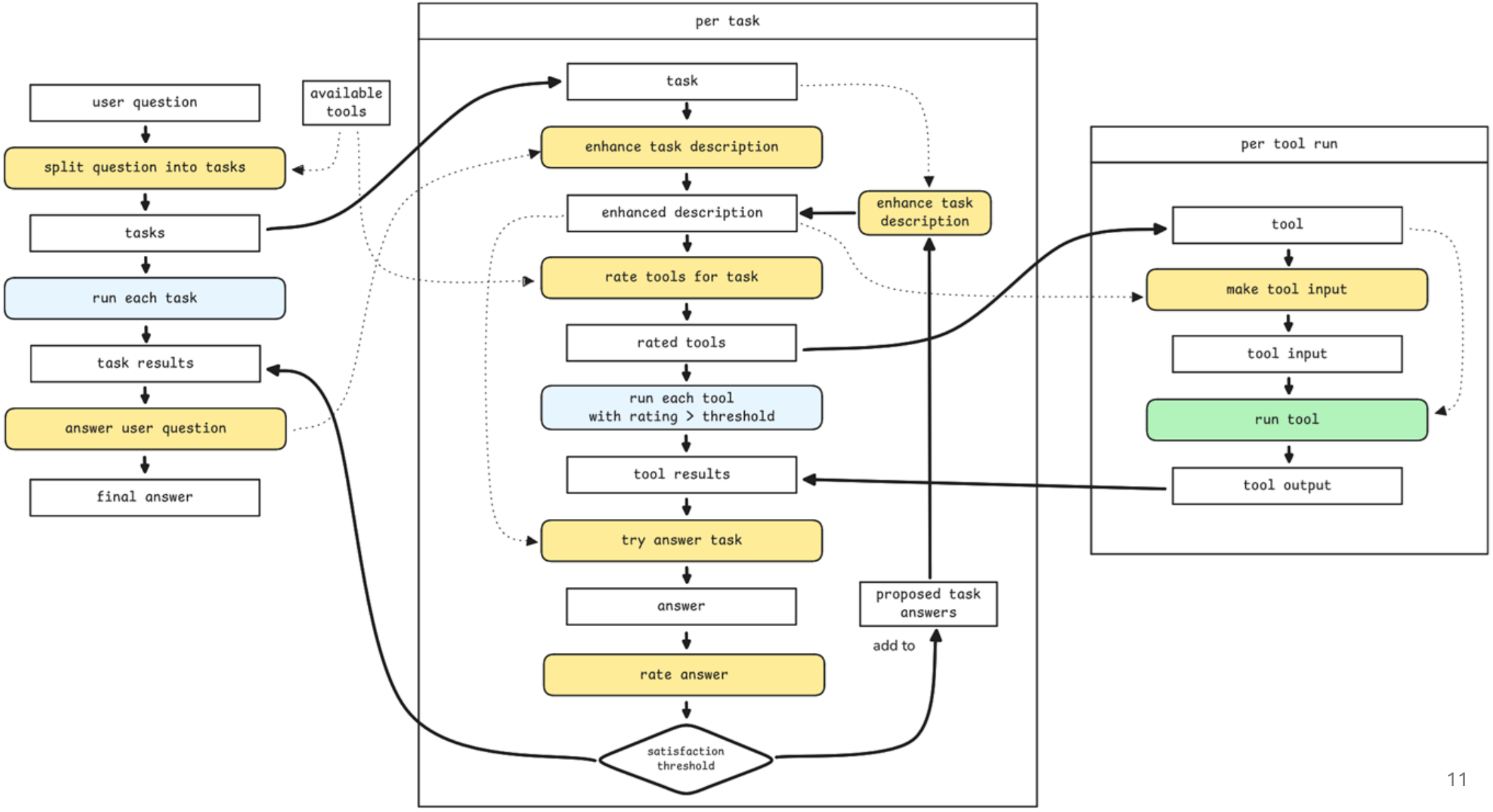
→ 657 SLOC, only dep. on Ollama

→ Ollama, 8B params, e.g. Qwen

Reactly: Task-splitting Workflow



Reactly: Architecture



Reactly: Implementation

```python
def run_query(user_question: str, history: List[ChatQA], tools: List[Tool], llm: LLM, agent_tracer: Tracer, agent_config: AgentConfig) -> str:
    query_tracer = agent_tracer.on("query", {'user_question': user_question})

    enhanced_user_question = enhance_user_question(user_question, history, llm, query_tracer)

    tasks: List[Task] = split_question_into_tasks(enhanced_user_question, tools, llm, query_tracer)
    task_results: List[TaskResult] = []

    for original_task in tasks:
        tracer = query_tracer.on("task", {'task': original_task.description})
        tool_results: List[ToolRunRecord] = []

        task = Task(enhance_task_description(original_task.description, task_results, llm, tracer))

        proposed_task_answers: List[TaskResult] = []
        for try_nr in range(agent_config.max_nr_tries_per_task):
            tracer.on('try', {'nr': try_nr})
            rated_tools = rate_tools_for_task(task, tools, llm, tracer)
            rated_tools.sort(key=lambda rt: -rt.score)

            for rated_tool in rated_tools:
                if rated_tool.score < agent_config.tool_use_rating_threshold:
                    break
                tool_input = make_tool_input(task, rated_tool.tool, llm, tracer)
                tool_result = run_tool(rated_tool.tool, tool_input, tracer)
                tool_results.append(ToolRunRecord(rated_tool.tool.name, tool_input, tool_result))

            task_answer = try_answer(task.description, tool_results, llm, tracer)
            satisfaction = rate_task_answer(task.description, task_answer, llm, tracer)

            if satisfaction >= agent_config.task_answer_satisfaction_threshold:
                task_results.append(TaskResult(task.description, task_answer, satisfaction))
                break
            else:
                proposed_task_answers.append(TaskResult(task.description, task_answer, satisfaction))
                task = Task(enhance_task_description(original_task.description, proposed_task_answers, llm, tracer))

    answer = str(try_answer(enhanced_user_question, [ToolRunRecord('subtask', t.task, t.result) for t in task_results], llm, query_tracer))
    query_tracer.on('result', {'result': answer})
    return answer
```

- 40 SLOC core control
- Supports tracing
- Only JSON to/from LLM
- Typed Python

Evaluation 1

HotspotQA Test trainrandom:0:100 (N=100)

	accuracy	HotpotQA				
		EM	F1	prec	recall	time
LLM: qwen2.5:7b-instruct-q8_0						
Reactly	64%	47%	56%	55%	62%	4.9sec
LLamaIndex ReAct	59%	35%	43%	42%	47%	1.2sec
	+8%	**+34%**	**+29%**	**+31%**	**31%**	**4x**
LLM: qwen2.5:3b-instruct-q4_K_M						
Reactly	10%	8%	10%	10%	9%	1sec
LLamaIndex ReAct	48%	38%	42%	42%	46%	1sec
	0.21x	**-80%**	**-77%**	**-76%**	**-80%**	**1x**

Evaluation 2

Custom-Logic+RAG on qwen2.5:7b-instruct-q8_0

	accuracy (ChatGPT)	HotpotQA				
		EM	F1	prec	recall	time
Test fox-easy:0:50 (N=50)						
Reactly	44%	16%	32%	28%	42%	3.8sec
LLamaIndex ReAct	26%	6%	14%	11%	24%	1.2sec
	+69%	**2.7x**	**2.4x**	**2.5x**	**+75%**	**3.2x**
Test fox-qa:0:20 (N=20)						
Reactly	35%	35%	35%	35%	35%	7.3sec
LLamaIndex ReAct	10%	10%	11%	11%	15%	1.3sec
	3.5x	**3.5x**	**3.1x**	**3.23x**	**2.3x**	**6x**

Live Demo

Conclusion: Lessons Learned

- Agentic AI is an active research field
- Getting LLM to reliable follow instruction & format is hard
- Always use simple JSON & “instruct” LLMs
- Larger improvement expected

Get access to project:

- Hosted on GitHub: github.com/d-volution/mlux-reactly
- Provided as PyPi package: mlux-reactly

References

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. Advances in neural information processing systems 30 (2017).
2. Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. Lost in the middle: How language models use long contexts, 2023. URL https://arxiv.org/abs/2307.03172 2 (2023).
3. Li, W., Lin, J., Jiang, Z., Cao, J., Liu, X., Zhang, J., Huang, Z., Chen, Q., Sun, W., Wang, Q., et al. Chain-of-agents: End-to-end agent foundation models via multi-agent distillation and agentic rl, 2025. URL https://arxiv.org/abs/2508.13167 2 (2025).
4. Del Rosario, R. F., Krawiecka, K., and de Witt, C. S. Architecting resilient llm agents: A guide to secure plan-then-execute implementations. arXiv preprint arXiv:2509.08646 (2025).
5. Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R. K.-W., and Lim, E.-P. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers) (2023), pp. 2609–2634.
6. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. Advances in neural information processing systems 35 (2022), 24824–24837
7. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. React: Synergizing reasoning and acting in language models. In The eleventh international conference on learning representations (2022). Zhang, Y., Sun, R., Chen, Y., Pfister, T., Zhang, R., and Arik, S. O. Chain of agents: Large language models collaborating on long-context tasks, 2024. URL https://arxiv.org/abs/2406.02818 3 (2024).